

Proof Assistants (Theorem Provers)

- Software tools that formally verify mathematical proofs and that assist in their construction
- What do they do?
 - Guide users in writing step-by-step formal proofs
 - Automatically check correctness of each step
 - Eliminate gaps, hidden assumptions, or “hand-wavy” reasoning
 - Provide a high level of confidence in results
- Key benefits
 - Absolute logical certainty: they catch subtle errors humans might miss
 - Reusable, machine-checkable libraries of formalised mathematics
 - Enable verification of complex mathematics and software systems
- Examples
 - Isabelle, Coq, Lean, Agda

Higher-Order Logic (HOL)

- The *Propositional Logic*:
 - no quantification, statements are atomic
 - can only combine facts using logical connectives
 - E.g. $\text{Socrates is human} \rightarrow \text{Socrates is mortal}$
- The *First-Order Logic* (FOL):
 - introduces variables and quantifies only over variables
 - can express general rules about objects and their properties
 - E.g. $\forall \text{person } x. x \text{ is human} \rightarrow x \text{ is mortal}$
- The *Higher-Order Logic* (HOL):
 - quantification also over functional or predicate symbols
 - can express statements about all possible properties
 - E.g. $\forall \text{property } P. P \text{ holds for Socrates} \rightarrow P \text{ holds for Plato}$

The Inductive Method

- An **inductively defined set** is a set whose elements are generated starting from one or more initial elements (base cases)
- New elements are obtained by repeatedly applying specific construction rules
- A typical example is the set of natural numbers:
 - **Base case**: 0 is natural
 - **Inductive case**: if n is natural then so is $\text{suc}(n)$
- The **inductive method** is a proof technique used to show that a property P holds for all elements of an **inductively defined set**

The Inductive Method - How it works

- For inductively defined sets, checking a property on each element individually is not feasible, since the set may be infinite
- To prove that a property P holds for all elements of the set, we proceed as follows
 - **Base cases**: prove that P holds for every initial element
 - **Inductive step**: for each construction rule, assume P holds for the input elements and prove it also holds for the newly generated element
- The key idea of induction is the following:
 - the property holds at the starting elements
 - the property is preserved by every construction rule
 - therefore, the property holds for all elements generated in the set



What is Isabelle?

- A generic interactive *proof assistant* for formal reasoning
- Supports multiple logical formalisms
- Based primarily on Higher-Order Logic (HOL)
- Combines human-guided proof development with built-in automated reasoning tools, such as:
 - **simp** launches the simplifier
 - **blast** launches the classical reasoner
 - **force** combines them
 - **auto** like force but applies to entire **proof state**, as we shall see
 - **clarify** does obvious steps, such as $P(X) \wedge Q(X)$ means both holds
- It can be downloaded and used free of charge
 - <https://isabelle.in.tum.de/installation.html>
- Offers a vast collection of certified and reusable knowledge
 - <https://www.isa-afp.org/>



How Isabelle Works

- Formal developments are organized into theories, which group definitions, lemmas, and proofs into modular files
- Proofs are constructed incrementally by the user, starting from basic facts and building more complex results step by step
- Each proof step is checked automatically by the system, ensuring logical correctness throughout the development
- The user interacts with Isabelle through Isabelle/jEdit, which provides continuous feedback and shows the proof state in real time

Why Isabelle - Paulson's Framework

- Introduced in 1998 as a formal framework for the analysis and verification of security protocols
- Its goal is to prove correctness properties of protocols, going beyond testing or attack discovery
- HOL is the language to specify a protocol and its goals
- The approach is inductive:
 - protocols are defined inductively
 - proofs are carried out using induction

Features of the protocol model

- The protocol is modeled as a set of possible executions, described in a formal way in HOL
 - A basic **event** represents a single action (e.g. sending a message)
 - A **trace** is one possible interleaving of events, possibly belonging to different protocol sessions, and including attacker's events
- It is defined inductively, by specifying:
 - initial traces (base cases)
 - rules to extend traces with new events
- The Dolev-Yao attacker (DY^+) is modeled explicitly through rules that allow the attacker to generate messages and extend the trace

Example

- Please only focus on structure of the 5 rules for now!!
 - One base case (obviously!) plus 4 inductive steps

Nil: $"[] \in flp"$

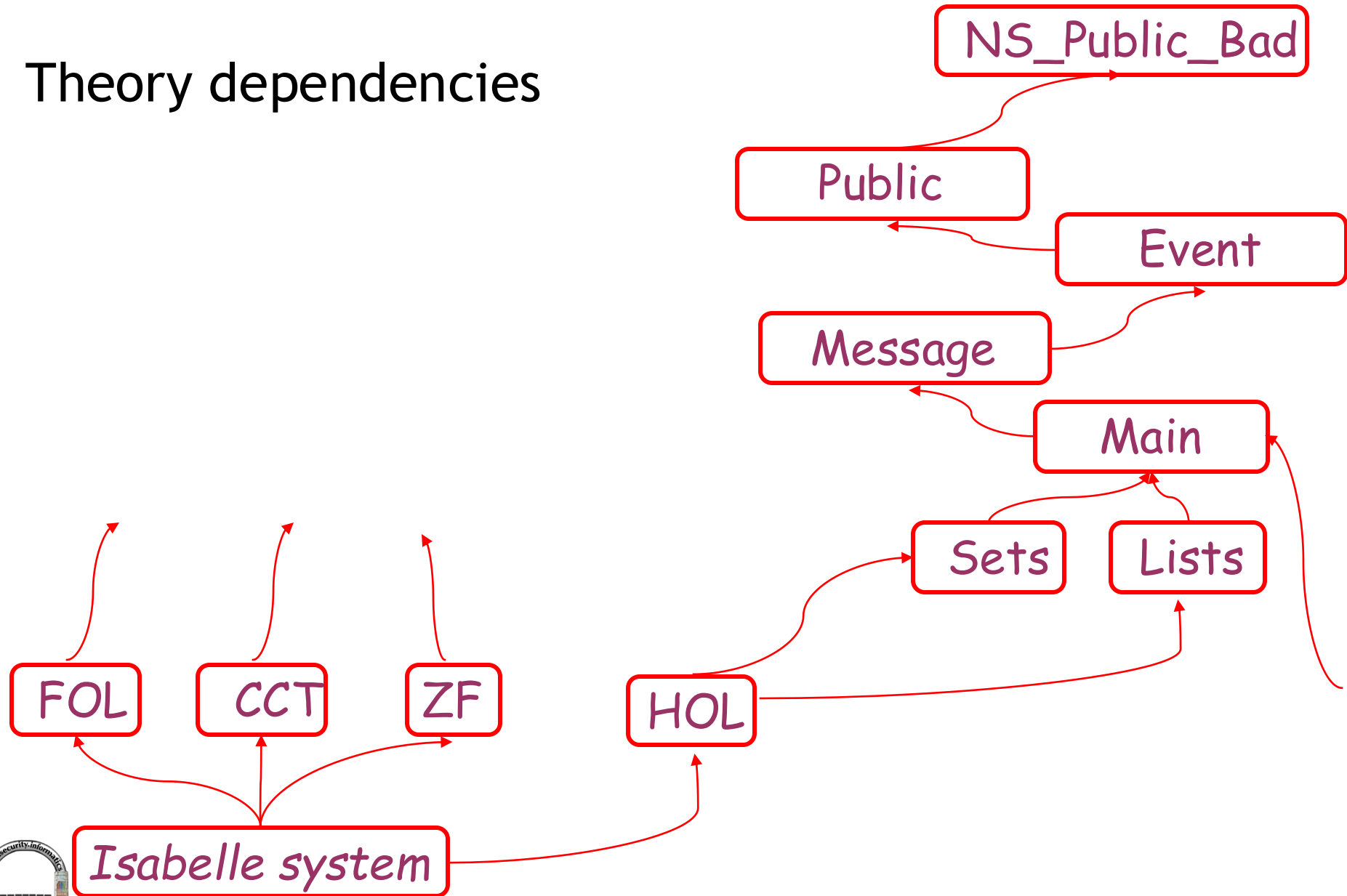
Fake: $"\llbracket evsf \in flp; X \in synth(analz(knows\ Spy\ evsf)) \rrbracket$
 $\implies Says\ Spy\ B\ X\ \# evsf \in flp"$

FLP1: $"\llbracket evs1 \in flp; Nonce\ Na \notin used\ evs1 \rrbracket$
 $\implies Says\ A\ B\ \{Agent\ A, Nonce\ Na\}\ \# evs1 \in flp"$

FLP2: $"\llbracket evs2 \in flp; Gets\ B\ \{Agent\ A, Nonce\ Na\} \in set\ evs2 \rrbracket$
 $\implies Says\ B\ A\ (Crypt\ (priSK\ B)\ (Nonce\ Na))\ \# evs2 \in flp"$

Recp: $"\llbracket evsr \in flp; Says\ A\ B\ X \in set\ evsr \rrbracket$
 $\implies Gets\ B\ X\ \# evsr \in flp"$

Theory dependencies



Main types: cryptographic keys

- Defined in src/HOL/Auth/Message.thy
- Key type is unbounded
- Function to invert a key, idempotent
- Symmetric keys remain the same when inverted

```
type_synonym
  key = nat

consts
  invKey      :: "key $\Rightarrow$ key"  — <inverse of a symmetric key>

specification (invKey)
  invKey [simp]: "invKey (invKey K) = K"

definition symKeys :: "key set" where
  "symKeys == {K. invKey K = K}"
```



Main types: agents

- Defined in src/HOL/Auth/Message.thy
- One TTP
- Unbounded number of protocol executors
- One attacker, Spy
- Datatype specifies the only type allowed type constructors
 - 3 in this case

```
datatype — <We allow any number of friendly agents>  
  agent = Server | Friend nat | Spy
```

Main types: messages

- Defined in src/HOL/Auth/Message.thy
- 7 type constructors
- Concatenated messages can be written nicely

datatype

msg = Agent	agent	—	<Agent names>
Number	nat	—	<Ordinary integers, <u>timestamps</u> , ...>
Nonce	nat	—	< <u>Unguessable nonces</u> >
Key	key	—	< <u>Crypto keys</u> >
Hash	msg	—	<Hashing>
MPair	msg msg	—	<Compound messages>
Crypt	key msg	—	<Encryption, public- or shared-key>

translations

"{x, y, z}" $\Rightarrow$ "{x, {y, z}}"
"{x, y}" $\Rightarrow$ "CONST MPair x y"

Main types: events

- Defined in src/HOL/Auth/Event.thy
- 3 main ones but others can be defined for specific protocols

datatype

```
event = Says agent agent msg  
       | Gets agent msg  
       | Notes agent msg
```

Example

- Here's a protocol and its inductive model
 - Note base case and reception case
 - Fake sets the protocol against DY⁺ attacker... how?

1. $A \rightarrow B : A, N_a$
2. $B \rightarrow A : \text{sign}(B, N_a)$

Nil: " $[] \in flp$ "

Fake: " $\llbracket evsf \in flp; X \in \text{synth}(\text{analz}(\text{knows Spy evsf})) \rrbracket$
 $\implies \text{Says Spy B X} \# evsf \in flp$ "

FLP1: " $\llbracket evs1 \in flp; \text{Nonce Na} \notin \text{used evs1} \rrbracket$
 $\implies \text{Says A B} \{ \text{Agent A, Nonce Na} \} \# evs1 \in flp$ "

FLP2: " $\llbracket evs2 \in flp; \text{Gets B} \{ \text{Agent A, Nonce Na} \} \in \text{set evs2} \rrbracket$
 $\implies \text{Says B A} (\text{Crypt} (\text{priSK B}) (\text{Nonce Na})) \# evs2 \in flp$ "

Recp: " $\llbracket evsr \in flp; \text{Says A B X} \in \text{set evsr} \rrbracket$
 $\implies \text{Gets B X} \# evsr \in flp$ "



An example trace

- Traces typically built from right to left, here bottom to top
- Built randomly
- Is it a trace of the protocol?

```
Nil: "[ ] ∈ flp"

Fake: "[[evsf ∈ flp; X ∈ synth(analz(knows Spy evsf))]]
      ⇒ Says Spy B X # evsf ∈ flp"

FLP1: "[[evs1 ∈ flp; Nonce Na ∉ used evs1]]
      ⇒ Says A B {|Agent A, Nonce Na|} # evs1 ∈ flp"

FLP2: "[[evs2 ∈ flp; Gets B {|Agent A, Nonce Na|} ∈ set evs2]]
      ⇒ Says B A (Crypt (priSK B) (Nonce Na)) # evs2 ∈ flp"

Recp: "[[evsr ∈ flp; Says A B X ∈ set evsr]]
      ⇒ Gets B X # evsr ∈ flp"
```

```
Says B C (Crypt (priSK B) (Nonce Na))
Gets B {|Agent A, Nonce Na|}
Says A B {|Agent A, Nonce Na|}
```


An example trace

- Is it a trace of the protocol?

Nil: " $[] \in flp$ "

Fake: " $\llbracket evsf \in flp; X \in synth(analz(knows\ Spy\ evsf)) \rrbracket$
 $\implies Says\ Spy\ B\ X\ \# evsf \in flp$ "

FLP1: " $\llbracket evs1 \in flp; Nonce\ Na \notin used\ evs1 \rrbracket$
 $\implies Says\ A\ B\ \{|Agent\ A, Nonce\ Na|\}\ \# evs1 \in flp$ "

FLP2: " $\llbracket evs2 \in flp; Gets\ B\ \{|Agent\ A, Nonce\ Na|\} \in set\ evs2 \rrbracket$
 $\implies Says\ B\ A\ (Crypt\ (priSK\ B)\ (Nonce\ Na))\ \# evs2 \in flp$ "

Recp: " $\llbracket evsr \in flp; Says\ A\ B\ X \in set\ evsr \rrbracket$
 $\implies Gets\ B\ X\ \# evsr \in flp$ "

Says B C (Crypt (priSK B) (Nonce Na))
Gets B {|Agent A, Nonce Na|}
Gets B {|Agent A, Nonce Na|}
Gets B {|Agent A, Nonce Na|}
Says A B {|Agent A, Nonce Na|}

An example trace

- Is it a trace of the protocol?

Nil: " $[] \in flp$ "

Fake: " $\llbracket evsf \in flp; X \in synth(analz(knows\ Spy\ evsf)) \rrbracket$
 $\implies Says\ Spy\ B\ X\ \#\ evsf \in flp$ "

FLP1: " $\llbracket evs1 \in flp; Nonce\ Na \notin used\ evs1 \rrbracket$
 $\implies Says\ A\ B\ \{Agent\ A, Nonce\ Na\}\ \#\ evs1 \in flp$ "

FLP2: " $\llbracket evs2 \in flp; Gets\ B\ \{Agent\ A, Nonce\ Na\} \in set\ evs2 \rrbracket$
 $\implies Says\ B\ A\ (Crypt\ (priSK\ B)\ (Nonce\ Na))\ \#\ evs2 \in flp$ "

Recp: " $\llbracket evsr \in flp; Says\ A\ B\ X \in set\ evsr \rrbracket$
 $\implies Gets\ B\ X\ \#\ evsr \in flp$ "

$Gets\ B\ \{|Agent\ A, Nonce\ Na|\}$

$Says\ A\ B\ \{|Agent\ A, Nonce\ Na|\}$

$Says\ B\ C\ (Crypt\ (priSK\ B)\ (Nonce\ Na))$

An example trace

- Is it a trace of the protocol?

Nil: " $[] \in flp$ "

Fake: " $\llbracket evsf \in flp; X \in synth(analz(knows\ Spy\ evsf)) \rrbracket$
 $\implies Says\ Spy\ B\ X\ \# evsf \in flp$ "

FLP1: " $\llbracket evs1 \in flp; Nonce\ Na \notin used\ evs1 \rrbracket$
 $\implies Says\ A\ B\ \{|Agent\ A, Nonce\ Na|\}\ \# evs1 \in flp$ "

FLP2: " $\llbracket evs2 \in flp; Gets\ B\ \{|Agent\ A, Nonce\ Na|\} \in set\ evs2 \rrbracket$
 $\implies Says\ B\ A\ (Crypt\ (priSK\ B)\ (Nonce\ Na))\ \# evs2 \in flp$ "

Recp: " $\llbracket evsr \in flp; Says\ A\ B\ X \in set\ evsr \rrbracket$
 $\implies Gets\ B\ X\ \# evsr \in flp$ "

Says B C (Crypt (priSK B) (Nonce Na))

Notes B {|Agent A, Nonce Na|}

Says A B {|Agent A, Nonce Na|}

An example trace

- Is it a trace of the protocol?

Nil: " $[] \in flp$ "

Fake: " $\llbracket evsf \in flp; X \in synth(analz(knows\ Spy\ evsf)) \rrbracket$
 $\implies Says\ Spy\ B\ X\ \# evsf \in flp$ "

FLP1: " $\llbracket evs1 \in flp; Nonce\ Na \notin used\ evs1 \rrbracket$
 $\implies Says\ A\ B\ \{Agent\ A, Nonce\ Na\}\ \# evs1 \in flp$ "

FLP2: " $\llbracket evs2 \in flp; Gets\ B\ \{Agent\ A, Nonce\ Na\} \in set\ evs2 \rrbracket$
 $\implies Says\ B\ A\ (Crypt\ (priSK\ B)\ (Nonce\ Na))\ \# evs2 \in flp$ "

Recp: " $\llbracket evsr \in flp; Says\ A\ B\ X \in set\ evsr \rrbracket$
 $\implies Gets\ B\ X\ \# evsr \in flp$ "



Main operators: parts

- Defined in src/HOL/Auth/Message.thy
- It extracts all components from a message set as if ciphertexts were broken
 - Hence the attacker cannot apply it!

inductive set

parts :: "msg set $\Rightarrow$ msg set"

for H :: "msg set"

where

Inj [intro]: " $X \in H \implies X \in \text{parts } H$ "

| Fst: " $\{X, Y\} \in \text{parts } H \implies X \in \text{parts } H$ "

| Snd: " $\{X, Y\} \in \text{parts } H \implies Y \in \text{parts } H$ "

| Body: " $\text{Crypt } K \ X \in \text{parts } H \implies X \in \text{parts } H$ "

Main operators: used

- Defined in src/HOL/Auth/Event.thy
- It serves to encode freshness
 - Hence if a nonce is unused on a trace then it is fresh on it
 - Gets case seems weird by think of protocol model...
- initState skipped though obvious

```
primrec used :: "event list  $\Rightarrow$  msg set"
where
  used_Nil:    "used []          = (UN B. parts (initState B))"
| used_Cons:  "used (ev # evs) =
  (case ev of
    Says A B X  $\Rightarrow$  parts {X}  $\cup$  used evs
  | Gets A X    $\Rightarrow$  used evs
  | Notes A X   $\Rightarrow$  parts {X}  $\cup$  used evs)"
```



An example trace

- Is it a trace of the protocol?

Nil: " $[] \in flp$ "

Fake: " $\llbracket evsf \in flp; X \in synth(analz(knows\ Spy\ evsf)) \rrbracket$
 $\implies Says\ Spy\ B\ X\ \# evsf \in flp$ "

FLP1: " $\llbracket evs1 \in flp; Nonce\ Na \notin used\ evs1 \rrbracket$
 $\implies Says\ A\ B\ \{|Agent\ A, Nonce\ Na|\}\ \# evs1 \in flp$ "

FLP2: " $\llbracket evs2 \in flp; Gets\ B\ \{|Agent\ A, Nonce\ Na|\} \in set\ evs2 \rrbracket$
 $\implies Says\ B\ A\ (Crypt\ (priSK\ B)\ (Nonce\ Na))\ \# evs2 \in flp$ "

Recp: " $\llbracket evsr \in flp; Says\ A\ B\ X \in set\ evsr \rrbracket$
 $\implies Gets\ B\ X\ \# evsr \in flp$ "

Says B C (Crypt (priSK B) (Nonce Na))
Gets B {|Agent A, Nonce Na|}
Says A B {|Agent A, Nonce Na|}
Says A' B' {|Agent A', Nonce Na|}

Main operators: analz

- Defined in src/HOL/Auth/Message.thy
- More realistically than parts, it opens ciphertexts coherently
 - Hence it makes sense to have the attacker apply it

```
inductive set
  analz :: "msg set  $\Rightarrow$  msg set"
  for H :: "msg set"
  where
    Inj [intro,simp]: " $X \in H \Rightarrow X \in \text{analz } H$ "
  | Fst: " $\{X,Y\} \in \text{analz } H \Rightarrow X \in \text{analz } H$ "
  | Snd: " $\{X,Y\} \in \text{analz } H \Rightarrow Y \in \text{analz } H$ "
  | Decrypt [dest]:
    " $\llbracket \text{Crypt } K \ X \in \text{analz } H; \text{Key}(\text{invKey } K) \in \text{analz } H \rrbracket \Rightarrow X \in \text{analz } H$ "
```


Main operators: synth

- Defined in src/HOL/Auth/Message.thy
- It forms messages by concatenation and encryption from available components
 - Hence it makes sense to have the attacker apply it

```
inductive_set
  synth :: "msg set => msg set"
  for H :: "msg set"
  where
    Inj      [intro]:  "X ∈ H ⇒ X ∈ synth H"
  | Agent   [intro]:  "Agent agt ∈ synth H"
  | Number  [intro]:  "Number n ∈ synth H"
  | Hash    [intro]:  "X ∈ synth H ⇒ Hash X ∈ synth H"
  | MPair   [intro]:  "[X ∈ synth H; Y ∈ synth H] ⇒ {X,Y} ∈ synth H"
  | Crypt   [intro]:  "[X ∈ synth H; Key(K) ∈ H] ⇒ Crypt K X ∈ synth H"
```

Basic lemmas

- All of these can be proved
 - $\text{analz } H$ included in parts H
 - $\text{analz}(\text{analz } H) = \text{analz } H$
 - $\text{synth}(\text{synth } H) = \text{synth } H$
 - $\text{analz}(\text{synth } H) = \text{analz } H \cup \text{synth } H$
 - $\text{Nonce } N \in \text{synth } H \Rightarrow \text{Nonce } N \in H$
 - $\text{Crypt } (K) X \in \text{synth } H \Rightarrow \text{Crypt } (K) X \in H \vee (X \in \text{synth } H \wedge K \in H)$
 - $\text{synth}(\text{analz } H)$ is the hardest to evaluate, skipped
 - ...
- Each lemma serves as rewriting rule for subsequent ones

Example

- One crucial function still remains to be defined... which one?

1. $A \rightarrow B : A, N_a$
2. $B \rightarrow A : \text{sign}(B, N_a)$

Nil: " $[] \in flp$ "

Fake: " $\llbracket evsf \in flp; X \in \text{synth}(\text{analz}(\text{knows Spy evsf})) \rrbracket$
 $\implies \text{Says Spy B X \# evsf} \in flp$ "

FLP1: " $\llbracket evs1 \in flp; \text{Nonce Na} \notin \text{used evs1} \rrbracket$
 $\implies \text{Says A B \{Agent A, Nonce Na\} \# evs1} \in flp$ "

FLP2: " $\llbracket evs2 \in flp; \text{Gets B \{Agent A, Nonce Na\}} \in \text{set evs2} \rrbracket$
 $\implies \text{Says B A (Crypt (priSK B) (Nonce Na)) \# evs2} \in flp$ "

Recp: " $\llbracket evsr \in flp; \text{Says A B X} \in \text{set evsr} \rrbracket$
 $\implies \text{Gets B X \# evsr} \in flp$ "

Main operators: knows

- Defined in src/HOL/Auth/Event.thy
- It implements knowledge of an agent over a given trace

```
consts
  bad      :: "agent set"      — <compromised agents>

specification (bad)
  Spy_in_bad      [iff]: "Spy ∈ bad"
  Server_not_bad [iff]: "Server ∉ bad"
```

Main operators: knows

```
primrec knows :: "agent  $\Rightarrow$  event list  $\Rightarrow$  msg set"
where
  knows_Nil:   "knows A [] = initState A"
| knows_Cons:
  "knows A (ev # evs) =
    (if A = Spy then
      (case ev of
        Says A' B X  $\Rightarrow$  insert X (knows Spy evs)
      | Gets A' X  $\Rightarrow$  knows Spy evs
      | Notes A' X  $\Rightarrow$ 
        if A'  $\in$  bad then insert X (knows Spy evs) else knows Spy evs)
    else
      (case ev of
        Says A' B X  $\Rightarrow$ 
        if A'=A then insert X (knows A evs) else knows A evs
      | Gets A' X  $\Rightarrow$ 
        if A'=A then insert X (knows A evs) else knows A evs
      | Notes A' X  $\Rightarrow$ 
        if A'=A then insert X (knows A evs) else knows A evs)))"
```

An example trace

- Is it a trace of the protocol?

Nil: " $[] \in flp$ "

Fake: " $\llbracket evsf \in flp; X \in synth(analz(knows\ Spy\ evsf)) \rrbracket$
 $\implies Says\ Spy\ B\ X\ \# evsf \in flp$ "

FLP1: " $\llbracket evs1 \in flp; Nonce\ Na \notin used\ evs1 \rrbracket$
 $\implies Says\ A\ B\ \{|Agent\ A, Nonce\ Na|\}\ \# evs1 \in flp$ "

FLP2: " $\llbracket evs2 \in flp; Gets\ B\ \{|Agent\ A, Nonce\ Na|\} \in set\ evs2 \rrbracket$
 $\implies Says\ B\ A\ (Crypt\ (priSK\ B)\ (Nonce\ Na))\ \# evs2 \in flp$ "

Recp: " $\llbracket evsr \in flp; Says\ A\ B\ X \in set\ evsr \rrbracket$
 $\implies Gets\ B\ X\ \# evsr \in flp$ "

Says B C (Crypt (priSK B) (Nonce Na))
Gets B {|Agent A, Nonce Na|}
Says A' B' {|Agent A', Nonce Na|}
Says A B {|Agent A, Nonce Na|}

An example trace

- Is it a trace of the protocol?

Nil: " $[] \in flp$ "

Fake: " $\llbracket evsf \in flp; X \in synth(analz(knows\ Spy\ evsf)) \rrbracket$
 $\implies Says\ Spy\ B\ X\ \# evsf \in flp$ "

FLP1: " $\llbracket evs1 \in flp; Nonce\ Na \notin used\ evs1 \rrbracket$
 $\implies Says\ A\ B\ \{|Agent\ A, Nonce\ Na|\}\ \# evs1 \in flp$ "

FLP2: " $\llbracket evs2 \in flp; Gets\ B\ \{|Agent\ A, Nonce\ Na|\} \in set\ evs2 \rrbracket$
 $\implies Says\ B\ A\ (Crypt\ (priSK\ B)\ (Nonce\ Na))\ \# evs2 \in flp$ "

Recp: " $\llbracket evsr \in flp; Says\ A\ B\ X \in set\ evsr \rrbracket$
 $\implies Gets\ B\ X\ \# evsr \in flp$ "

Says B C (Crypt (priSK B) (Nonce Na))
Gets B {|Agent A, Nonce Na|}
Says Spy B {|Agent A, Nonce Na'|}
Says A B {|Agent A, Nonce Na|}

An example trace

- Is it a trace of the protocol?

Nil: " $[] \in flp$ "

Fake: " $\llbracket evsf \in flp; X \in synth(analz(knows\ Spy\ evsf)) \rrbracket$
 $\implies Says\ Spy\ B\ X\ \# evsf \in flp$ "

FLP1: " $\llbracket evs1 \in flp; Nonce\ Na \notin used\ evs1 \rrbracket$
 $\implies Says\ A\ B\ \{|Agent\ A, Nonce\ Na|\}\ \# evs1 \in flp$ "

FLP2: " $\llbracket evs2 \in flp; Gets\ B\ \{|Agent\ A, Nonce\ Na|\} \in set\ evs2 \rrbracket$
 $\implies Says\ B\ A\ (Crypt\ (priSK\ B)\ (Nonce\ Na))\ \# evs2 \in flp$ "

Recp: " $\llbracket evsr \in flp; Says\ A\ B\ X \in set\ evsr \rrbracket$
 $\implies Gets\ B\ X\ \# evsr \in flp$ "

Says B C (Crypt (priSK B) (Nonce Na))

Gets B {|Agent A, Nonce Na|}

Says Spy B {|Agent A, Nonce Na|}

Encoding the threat model

- The set of all messages that the attacker can synthesize from the analysis of her observation of the traffic on a given trace `evsf` is defined as `synth(analz(knows Spy evsf))`
- Hence the Fake rule, present in each protocol model, introduces an attacker who sends out any of the messages she can fake, as described above, to anyone
- Clearly, no bound stated on the attacker's capabilities, contrarily to other approaches, such as state enumeration
- This is perhaps the main strength of the Inductive Method
- We skip the symbolic analysis of `synth(analz(knows Spy evsf))`, which does have a set that bounds it, contrarily to intuition, handled automatically by the dedicated proof method `spy_analz` whose explanation is a separate course!



Encoding the main protocol goals

- Secrecy of m encoded as: $\forall$ trace evs of the protocol model, it must be the case that m does not belong to what the attacker can analyse from her knowledge over evs
 - In practice evs is a free variable in the model, i.e. a generic trace of the model
 - I.e. $evs \in flp.$ Key $K \notin \text{analz}(\text{knows Spy } evs)$
- Authentication of A with B encoded as: $\exists$ message m such that, $\forall$ trace evs of the protocol model on which B receives m , it must have been A who sent m to B on the trace
 - In practice evs is a free variable in the model, i.e. a generic trace of the model
 - I.e. $evs \in flp.$ If Gets B m appears on evs , then also Says A B m does
- Warning: notation still much simplified in this page!

Towards the proofs

- Safety properties: prove that in every event of every trace, the stated security goal holds
- More precisely, two forms of induction
 - Usual form for $\forall n \in \text{Nat}. P(n)$
 - Base case: $P(0)$
 - Induction step: $P(x) \Rightarrow P(x+1)$
 - Conclusion: $\forall n \in \text{Nat}. P(n)$
 - Minimal counterexample form
 - Conjecture: $\exists x. \neg P(x) \wedge \forall y < x. P(y)$
 - Prove contradiction
 - Conclusion: $\forall n \in \text{Nat}. P(n)$
- Typical proof scheme: build inductive formula, apply induction, simplify and, if needed, apply classical reasoner

Rules of thumb

- Build a conjecture to attempt to prove (to see if it is a theorem) by informally checking if each rule defining the protocol model preserves it
- After applying induction, be prepared to a number of subgoals in the proof state equal to the number of rules defining the protocol model
 - Hence the generic trace is the rule is conveniently numbered!
- Failing to prove a conjecture either
 - Means that you must concentrate more ☺
 - Or that you are facing a proof state that cannot be solved
 - It should be possible to understand why, what makes the conjecture fail
 - If your conjecture was a basic goal, then you probably found an attack!

An example theorem

- Observe theorem statement and theorem proof script
- Each “apply” applies a method - let’s explain the full syntax
- The proof script fails to show how the actual proof unfolding

- It can be seen live!

theorem *Spy-not-see-NA*:

$\llbracket \text{Says } A \ B \ (\text{Crypt}(\text{pubEK } B) \ \{\text{Nonce } NA, \text{Agent } A\}) \in \text{set } \text{evs};$
 $A \notin \text{bad}; \ B \notin \text{bad}; \ \text{evs} \in \text{ns-public} \rrbracket$
 $\implies \text{Nonce } NA \notin \text{analz} \ (\text{spies } \text{evs})$

apply (erule rev-mp, erule ns-public.induct)

apply simp-all

apply (rule conjI, clarify)

apply clarify

apply (drule Fake-analz-insert [THEN subsetD], simp)

apply simp

apply (blast dest: unique-NA intro: no-nonce-NS1-NS2)+
done



Object vs. meta level

- Object level is the level of the specification language
 - HOL in our case, which is also addressed as object logic
- Meta level is the level of the abstract reasoning
- Example 1 (introducing meta-level implication)
 - Suppose you know that $P(X) \vee Q(X)$
 - Then I also tell you that $\neg P(X)$
 - So you deduce that $Q(X)$
 - The entire reasoning is written $((P(X) \vee Q(X)) \wedge \neg P(X)) \Rightarrow Q(X)$
- Example 2 (using both implications)
 - Suppose you know that $P(X) \rightarrow Q(X)$
 - Then I also tell you that $P(X)$
 - So you deduce $Q(X)$
 - The entire reasoning is written $((P(X) \rightarrow Q(X)) \wedge P(X)) \Rightarrow Q(X)$

Demo!

- Interacting with Isabelle
 - Through Message.thy
 - Then Event.thy
 - Then Public.thy
 - And finally NS_Public_Bad.thy with its failed proof attempt of authentication of Alice with Bob, which indicates Lowe's attack